

Audit

1. Feature Overview

Staff **Audit** screen and APIs provide:

1. **Activity event trail** (`audit_event`) — categorized actions across AUTH, CUSTOMER, ACCOUNT, TRANSFER, PORTAL, STAFF, ROLE, POLICY.
2. **Transfer approval history** — resolved portal transfer requests (approve / reject / executed).
3. **Historical backfill** — rebuild missing events from existing domain tables (Admin only). Idempotent; safe to re-run.

Login/logout **after** deploy are recorded under category `AUTH` . Past logins cannot be recovered (never stored historically).

Status: Implemented (activity trail + approval history + backfill)

2. Business Purpose

Compliance and investigation: who did what, on which target, when. Separate **Login / Logout** from business mutations.

3. User Workflow

1. Login as ADMIN, MANAGER, or AUDITOR.
2. Sidebar → **Audit**.
3. Filter by category tabs (All, Login/Logout, Customer, Account, ...) and optional actor username.
4. **Approval history** tab → resolved transfer approvals detail.
5. Admin may click **Backfill history** once after upgrade.

4. Execution Flow

Live write

```
DomainService.mutatingOp()  
    ↓  
AuditEventService.record(...)    // REQUIRES_NEW, best-effort  
    ↓  
audit_event row
```

Login / logout

```
AuthenticationService.login / logout  
    ↓  
AuditEventService.recordForUser / record    (AUTH)
```

Backfill

```
POST /audit/backfill
  ↓
AuditBackfillService.backfill()
  ↓
Read customers, accounts, transactions, transfer_request,
beneficiary, users, roles → insert missing keys only
```

5. Database Tables

audit_event

Entity: `com.bankone.audit.entity.AuditEventEntity`

Column	Notes
<code>id</code>	PK
<code>category</code>	Enum string: AUTH, CUSTOMER, ACCOUNT, TRANSFER, STAFF, ROLE, POLICY, PORTAL
<code>action</code>	Stable code (<code>AuditAction</code>), e.g. LOGIN, ACCOUNT_DEPOSIT, TRANSFER_APPROVED
<code>actor_username</code>	Who performed the action (nullable for system / backfill)
<code>actor_user_id</code>	Optional user id
<code>target_type</code> / <code>target_id</code>	What was touched (CUSTOMER, ACCOUNT, TRANSACTION, ...)
<code>summary</code> / <code>details</code>	Human text + optional details
<code>success</code>	false for LOGIN_FAILED etc.
<code>created_at</code>	Event time (historical for backfill)

Indexes: `created_at` , `category` , `actor_username` .

6. REST APIs

Base: `/audit` — roles **ADMIN** | **MANAGER** | **AUDITOR**

Method	Path	Notes
GET	<code>/audit/events</code>	Page of events. Query: <code>category</code> , <code>action</code> , <code>actor</code> , <code>page</code> , <code>size</code>
GET	<code>/audit/transfer- F approvals</code>	ull resolved approval list
POST	<code>/audit/backfill</code>	ADMIN only. Rebuild missing historical events. Returns <code>{ inserted, skipped, insertedBySource }</code>

Also: `POST /auth/logout` records AUTH LOGOUT (authenticated).

7-13. Code map

Layer	Classes
Controller	<code>AuditController</code>
Service	<code>AuditEventService</code> , <code>AuditBackfillService</code>
Domain	<code>AuditCategory</code> , <code>AuditAction</code>
Entity / DTO	<code>AuditEventEntity</code> , <code>AuditEventRepository</code> , <code>AuditEventResponse</code> , <code>AuditBackfillResult</code>
Frontend	<code>features/audit/*</code> , <code>PortalService.listAuditEvents</code> , <code>backfillAuditHistory</code>

Instrumentation sites (examples): `AuthenticationService` , `CustomerServiceImpl` , `AccountServiceImpl` , `TransferApprovalService` , `PortalTransferService` , `BeneficiaryService` , `UserService` , `RoleService` , `AccountPolicyServiceImpl` .

14. Validation Rules

Backfill skips rows whose `(action, target_type, target_id)` already exist. Page `size` capped at 100 in search.

15. Security Rules

- Read audit: ADMIN, MANAGER, AUDITOR
- Backfill: ADMIN only
- Never public

16. Exception Handling

Live `record()` swallows write failures (log warn) so business TX is not failed by audit. Backfill failures surface as API 500 (fixed lazy-load via join-fetch).

17. Logging

App logs ≠ durable audit. Prefer `audit_event` for investigations.

18. Audit Events (categories)

Category	Examples
AUTH	LOGIN, LOGIN_FAILED, LOGOUT, CHANGE_PASSWORD
CUSTOMER	CUSTOMER_CREATE, CUSTOMER_UPDATE
ACCOUNT	ACCOUNT_OPEN, DEPOSIT, WITHDRAW, TRANSFER, STATUS_CHANGE
TRANSFER	TRANSFER_APPROVED, TRANSFER_REJECTED
PORTAL	TRANSFER_REQUESTED, PORTAL_TRANSFER, BENEFICIARY_*, portal USER_CREATE
STAFF	USER_CREATE / UPDATE (employees)
ROLE	ROLE_CREATE / UPDATE
POLICY	POLICY_UPDATE

19. Testing Strategy

- Login → AUTH event
- Deposit → ACCOUNT_DEPOSIT
- Backfill twice → second run `inserted=0`, high `skipped`
- Non-admin POST backfill → 403

20. Future Extension Guide

See [TECH_LEARNING_PLAN.md](#): correlation IDs on events, cursor pagination, purge job for old rows.

Future Modification Guide

Requirement: Add correlation id to every event

Item	Detail
Files	New filter + <code>AuditEventEntity.correlationId</code> ; <code>AuditEventService.record</code>
Impact	Observability; optional Kafka header reuse

Call hierarchy (live deposit)

```
AccountServiceImpl.deposit()  
    ↓  
AuditEventService.record(ACCOUNT, ACCOUNT_DEPOSIT, ...)  
    ↓  
AuditEventRepository.save()
```